Введение. Принципы построения приложения

Table of contents

Введение. Принципы построения приложения . . . 1
Компоненты . . . 1
Компонент как класс . . . 1
Слои и зависимости . . . 3
Принципы SOLID . . . 5
Внедрение зависимостей . . . 5

Введение. Принципы построения приложения

Первая лекция закладывает фундамент: **что такое компонент и как компоненты зависят друг от друга?**

Компонент — это базовая единица вычисления или хранения данных с чётким контрактом (интерфейсом) и набором зависимостей. Это может быть класс, модуль, библиотека или микросервис. Главное — у него есть чёткая граница: что он принимает на вход, что возвращает, от чего зависит.

Управление зависимостями — ключевая задача архитектуры. Если компонент A зависит от компонента B, то изменения в B могут сломать A. Когда в проекте сотни компонентов, неконтролируемые зависимости превращают код в «клубок спагетти», где невозможно понять, что от чего зависит.

Лекция вводит понятия **слоя** (множество компонентов с общей ответственностью) и **правил межслойных зависимостей** (например, «слой бизнес-логики не должен зависеть от слоя UI»). Эти правила помогают избежать циклических зависимостей и упрощают тестирование.

Компонент

Компоненты

Рассмотрим несколько примеров:

Компонент как класс

```
#include <iostream>

class NumberPrinter {
public:
    void print(int x) {
        std::cout << x << std::endl;
```

```
    }

    void printRange(int from, int to) {
        for (int i = from; i <= to; ++i) {
            std::cout << i << std::endl;
        }
    }
};
```

Единица вычисления:	форматирование и вывод чисел
Интерфейс:	сигнатуры публичных методов
Зависимость:	консоль ввод-вывод
Форма реализации:	класс (ООП-модель)

- Компонент как функция (Elixir)

```
def send_message(host, port, message) do
  {:ok, socket} = :gen_tcp.connect(host, port, [:binary, active: false])
  :ok = :gen_tcp.send(socket, message)
  :gen_tcp.close(socket)
end
```

Единица вычисления:	передача сообщения
Интерфейс:	сигнатура функции
Зависимость:	сеть (TCP stack)
Форма реализации:	функция (функциональная модель)

- Компонент как функция (Go)

```
func LoadUserNames(db *sql.DB) ([]string, error) {
    rows, err := db.Query("SELECT name FROM users")
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var result []string
    for rows.Next() {
        var name string
        if err := rows.Scan(&name); err != nil {
```

```go
            return nil, err
        }
        result = append(result, name)
    }
    return result, nil
}
```

Единица вычисления:	извлечение данных
Интерфейс:	сигнатура функции
Зависимость:	БД (SQL, соединение)
Форма реализации:	функция (процедурная / Go-модель)

Слои и зависимости

Слой

Рассмотрим пример слоя персистентности, который сохраняет различные модели в CSV:

```go
// domain/models.go - слой моделей данных
package domain

type User    struct{ ID int; Name, Email string }
type Item    struct{ ID int; Title string; Price float64 }
type Storage struct{ ID int; Location string; Capacity int }
```

```go
// persistence/csv_persistence.go - слой персистентности
package persistence

import (
    "fmt"
    "os"
    "strings"

    "example/domain"  // Явная зависимость от слоя моделей
)

func SaveUsersToCSV(users []domain.User, filename string) error {
    lines := make([]string, 0, len(users))
    for _, u := range users {
        lines = append(lines, fmt.Sprintf("%d,%s,%s", u.ID, u.Name, u.Email))
    }
    return write(filename, lines)
```

```go
}

func SaveItemsToCSV(items []domain.Item, filename string) error {
    lines := make([]string, 0, len(items))
    for _, i := range items {
        lines = append(lines, fmt.Sprintf("%d,%s,%v", i.ID, i.Title, i.Price))
    }
    return write(filename, lines)
}

func SaveStoragesToCSV(storages []domain.Storage, filename string) error {
    lines := make([]string, 0, len(storages))
    for _, s := range storages {
            lines = append(lines, fmt.Sprintf("%d,%s,%d", s.ID, s.Location,
s.Capacity))
    }
    return write(filename, lines)
}

func write(filename string, lines []string) error {
    return os.WriteFile(filename, []byte(strings.Join(lines, "\n")), 0644)
}
```

Анализ слоёв:

Слой	Компонент	Контракт (интерфейс)	Зависимости
domain	User	struct{ ID int; Name, Email string }	—
domain	Item	struct{ ID int; Title string; Price float64 }	—
domain	Storage	struct{ ID int; Location string; Capacity int }	—
persistence	SaveUsersToCSV	([]domain.User, string) error	domain.User, ФС
persistence	SaveItemsToCSV	([]domain.Item, string) error	domain.Item, ФС
persistence	SaveStoragesToCSV	([]domain.Storage, string) error	domain.Storage, ФС

Слой	Компонент	Контракт (интерфейс)	Зависимости
`persistence`	`write`	`(string, []string) error`	ФС

Заметим: - **Слой моделей** (`domain`): три компонента-структуры без зависимостей - **Слой персистентности** (`persistence`): четыре компонента-функции, три из которых явно зависят от типов слоя моделей - Слой персистентности **зависит** от слоя моделей, но не наоборот

Принципы SOLID

SOLID — набор принципов проектирования, уменьшающих связанность кода и повышающих локальность изменений.

- **Принцип единственной ответственности (SRP):** у компонента должна быть одна причина для изменения.
- **Принцип открытости/закрытости (OCP):** поведение расширяется через новые реализации, а не через модификацию стабильного кода.
- **Принцип подстановки Лисков (LSP):** подтип должен корректно заменять базовый тип без изменения ожидаемого поведения.
- **Принцип разделения интерфейсов (ISP):** лучше несколько узких интерфейсов, чем один «толстый» универсальный.
- **Принцип инверсии зависимостей (DIP):** бизнес-логика зависит от абстракций, а не от конкретных деталей инфраструктуры.

В контексте SOLID интерфейс удобно понимать как **контракт взаимодействия**: он фиксирует, *что* компонент обязан предоставить, но не навязывает, *как* это реализовано. Форма интерфейса зависит от языка: - **Go:** отдельный тип `interface` с набором методов; - `Java/C#` — явная декларация `interface` и классы-реализации; - **Python:** обычно протокол поведения через абстрактные базовые классы (`abc`) или структурные протоколы (`Protocol`).

Практический эффект SOLID: изменение формата хранения данных или транспортного протокола затрагивает ограниченное число компонентов, а не всю систему.

Внедрение зависимостей

После того как в системе появляется много компонентов, ключевым становится вопрос управления зависимостями между ними. Внедрение зависимостей (Dependency Injection, DI) — это способ передачи компоненту его зависимостей извне, а не создания их внутри компонента.

DI-контейнер — инфраструктурный механизм, который: - регистрирует реализации компонентов; - строит и разрешает граф зависимостей; - создаёт экземпляры в правильном порядке (с учётом времени жизни объектов).

По способу разрешения зависимостей обычно различают два подхода: - **На этапе компиляции (compile-time DI):** граф зависимостей формируется заранее, ошибки связей обнаруживаются до запуска приложения. - **Во время выполнения (run-time DI):** контейнер разрешает зависимости при запуске, что даёт больше гибкости конфигурации, но переносит часть ошибок на момент старта/работы системы.

Без DI компонент жёстко связан с конкретными реализациями (например, конкретной СУБД или конкретным HTTP-клиентом), что усложняет тестирование и замену инфраструктурных модулей. С DI компонент работает через интерфейсы, а конкретные реализации подключаются в точке композиции приложения (`composition root`).

Простейший пример конструкторной инъекции:

```go
type UserRepo interface {
  FindByID(id int) (User, error)
}

type UserService struct {
  repo UserRepo
}

func NewUserService(repo UserRepo) *UserService {
  return &UserService{repo: repo}
}

func BuildApp() *UserService {
  db := NewPostgresDB()
  repo := NewPostgresUserRepo(db)
  return NewUserService(repo) // сборка графа зависимостей
}
```

На небольших проектах такой граф зависимостей часто собирают вручную. По мере роста системы это становится трудоёмко: увеличивается число компонентов, ветвлений конфигурации и рисков ошибиться в порядке инициализации. Поэтому на практике используют готовые библиотеки: - **Go:** `uber-go/dig`, `google/wire`; - **Java/Kotlin:** Spring Container, Guice, Dagger; - **.NET:** встроенный контейнер `Microsoft.Extensions.DependencyInjection`, Autofac; - **Python:** `dependency-injector`, `Dishka`; - **Node.js/TypeScript:** InversifyJS, NestJS DI.

Отдельный класс нюансов связан с временем жизни объектов. Обычно различают: - **Singleton** (один экземпляр на приложение), - **Scoped** (один экземпляр на запрос/операцию), - **Transient** (новый экземпляр при каждом запросе зависимости).

Точные правила объявления и разрешения времени жизни зависят от конкретной библиотеки контейнера.

Практические правила: - создавать зависимости на границе приложения, а не внутри бизнес-методов; - передавать зависимости через конструктор или параметры инициализации; - не превращать DI-контейнер в «глобальный сервис-локатор».

Результат: проще модульные тесты, дешевле замена инфраструктуры, прозрачнее граф зависимостей.